

TILEWEAVE: Programming Across Abstraction Levels for Coding Agents on Accelerator Kernels

ANONYMOUS AUTHOR(S)

Programming languages and abstractions have traditionally been designed for humans, but code generation is increasingly performed by models and autonomous coding agents. The abstractions that best support human programmers may therefore not be optimal for machine-driven programming. We investigate this question through GPU and AI-accelerator kernel generation, where different abstraction levels expose complementary trade-offs: high-level domain-specific languages (DSLs) simplify search by delegating implementation details to compilers, while lower-level representations provide finer-grained control for more precise performance tuning at the cost of a larger and more complex search space.

In this paper, we present TILEWEAVE, a unified, pluggable interface that enables coding agents to develop kernels across multiple editable abstraction levels. Rather than committing to a single representation, TILEWEAVE allows an agent to transfer and refine candidates across levels through *Cross-Level Diagnosis*, which maps correctness and performance feedback to actionable controls at different abstraction levels; *Abstraction Scheduling*, which selects both the candidate and representation from which subsequent refinement should proceed; and a *Heterogeneous Search Tree*, which preserves and revisits alternative search trajectories spanning multiple representations.

We integrate TILEWEAVE with two code agents AKO and KDA, and evaluate both integrations on 50 end-to-end KernelBench Level 3 workloads across two accelerator platforms and two language models. Giving equal weight to the four accelerator-model configurations, AKO with TILEWEAVE achieves an arithmetic-mean speedup of 4.84 $\times$, compared with 3.75 $\times$ for the strongest evaluated baseline without TILEWEAVE, an improvement of 29.0%. Integrating the same interface with KDA improves its mean speedup by 69.2%, with gains in every configuration. These results show that allowing coding agents to navigate across programming abstractions can improve machine-driven software development and can be reused across different base workflows.

CCS Concepts: • **Software and its engineering** → **Software development methods**; *Compilers*; • **Computing methodologies** → *Artificial intelligence*.

Additional Key Words and Phrases: abstraction levels, accelerator kernels, cross-level diagnosis, abstraction-level routing, code agents

1 Introduction

As coding agents take on more code generation tasks, understanding how programming abstractions shape code development becomes increasingly important. Google reports that 75% of its new code is generated by artificial intelligence (AI) systems and approved by engineers [46]. Anthropic reports that, as of May 2026, more than 80% of its merged code was authored by Claude [16]. This shift raises a fundamental question: are abstractions designed for human programmers also the most effective interfaces for coding agents? Because agents develop programs through repeated generation, evaluation, and refinement, the choice of abstraction directly shapes the search space that the agents and models must navigate.

Accelerator kernel programming provides a particularly useful setting for studying this question because performance depends on decisions spanning multiple abstraction levels. High-level domain-specific languages (DSLs), such as Triton [57] and TileLang [61], provide compact operations for computation and data movement while delegating some implementation details to the compiler. In contrast, low-level native source code exposes more direct control over memory accesses, synchronization, and hardware operations, enabling optimizations unavailable at higher levels but requiring the agent to reason about a substantially larger set of interacting implementation choices.

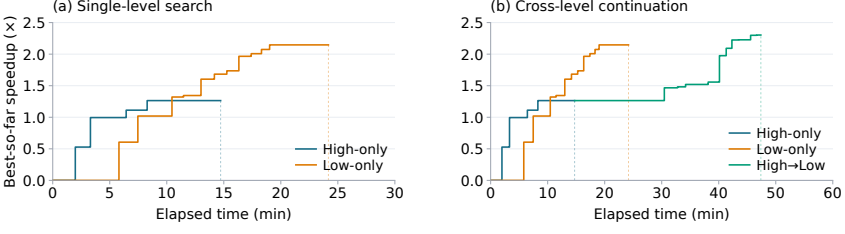


Fig. 1. Performance trajectories for MinGPT on an AMD Radeon AI PRO R9700. High and Low denote TileLang and HIP C++, respectively. (a) High-only search achieves larger speedups early in the search, while Low-only search reaches a higher speedup later. (b) Continuing development at the low level from an exported high-level candidate yields further improvement.

These properties make different abstraction levels useful at different stages of machine-driven software development. Early in the search, a high-level representation can help an agent rapidly explore algorithmic structures and reach a strong implementation without managing every hardware detail. Once progress at that level saturates, however, further improvement may require controls exposed only by a lower-level representation. Figure 1(a) illustrates this behavior: High-only improves more rapidly early in the search, whereas Low-only progresses slowly but with much greater cost and will eventually reach a higher performance level. By transferring a promising high-level candidate into editable low-level code, an agent can preserve the progress already made while opening a new space of optimization opportunities; Figure 1(b) shows the further improvement obtained through continued low-level development in the same example.

This observation motivates *cross-level programming*: rather than committing to a single representation throughout development, an agent should be able to move across abstraction levels as the search evolves. Existing agentic kernel workflows [42, 66] provide the loop of candidate generation, evaluation, profiling, and iterative refinement; we refer to this underlying process as the *base workflow*. Cross-level programming extends this process by allowing refinement to continue in different representations, each exposing different controls and search spaces. The key challenge is therefore not merely supporting multiple languages, but coordinating when and how the search should move across abstraction levels while retaining useful information from prior exploration.

Such coordination requires three related decisions: *what* should be changed (*i.e.*, identifying editable controls relevant to the feedback), *where* the change should be made (*i.e.*, selecting a level for the proposed modification), and *which prior candidate* should serve as the starting point (to preserve the context of previous observations). First, hardware evidence, such as execution timings and available profiling data, identifies observed performance behavior, but does not directly reveal the programming decisions responsible for the behavior. A low-level bottleneck may originate from high-level choices in data layout or computation structure. The agent must therefore relate hardware evidence back to editable controls in the program. Second, once a modification goal is identified, the same modification goal may be realizable at multiple levels, with different degrees of control, implementation complexity, and correctness risks. The agent must determine which level provides the most appropriate interface for the next refinement. Third, these assessments also depend on the history of each candidate, because edits and representation conversions can change the conditions under which previous observations apply. Cross-level programming must therefore preserve candidate histories so that each observation remains associated with the context in which it was obtained.

To address these challenges, we present TILEWEAVE, a pluggable method for feedback-guided programming of accelerator kernels across abstraction levels. Given candidate code and evaluation

feedback from the base workflow, TILEWEAVE compares development options across abstraction levels using the available evidence and candidate history. TILEWEAVE returns the selected action, source candidate, and target abstraction level to guide the base workflow’s next code generation or modification step. We propose three cooperating components to support TILEWEAVE, as shown in Figure 2. ① *Cross-Level Diagnosis* establishes source correspondence between a candidate’s code representations. Profile association links available hardware evidence to the executed code of the corresponding candidate, helping the next component examine the implementation choices behind the observed behavior and identify relevant editable controls. ② *Abstraction Scheduling* uses the evidence to develop modification plans, compare available controls and implementation requirements across levels, and combine these comparisons with candidate history to select a source candidate and target abstraction level. Possible choices include further edits at the current level, a supported conversion to a lower level, and renewed exploration from a retained higher-level candidate. ③ *Heterogeneous Search Tree* maintains the relationships between candidates created by code modifications and representation conversions, together with evaluation results for retained candidates. These records preserve the context of each observation, helping other components assess the relevance of prior observations when selecting a candidate for further modification.

We evaluate TILEWEAVE using two recent agentic kernel workflows, Agentic Kernel Optimization (AKO) [66] and Kernel Design Agents (KDA) [42]. Both workflows use profiling feedback and candidate history to guide iterative kernel development. Our experiments cover 50 model workloads from KernelBench Level 3 (full machine learning architectures) [44] across two accelerator platforms (Ascend [35] and AMD [3]) and two language models (GPT-6 Astra [43] and DeepSeek-V4.1-Flash [13]). We compare AKO+TILEWEAVE against single-level baselines and alternative cross-level strategies, and use KDA+TILEWEAVE to assess reusability across base workflows. Across four accelerator–model configurations, AKO+TILEWEAVE improves the equal-weight overall mean speedup by 29.0% over the strongest baseline by overall mean. Integrating TILEWEAVE with KDA improves KDA’s arithmetic mean speedup by 69.2%.

In this paper, we make the following main contributions:

- We propose a feedback-guided method that can jointly select source candidates and target abstraction levels for continued kernel development.
- We design and implement TILEWEAVE as a pluggable extension of existing agentic kernel workflows.
- We systematically evaluate TILEWEAVE across multiple accelerator platforms and language models against single-level baselines and alternative cross-level strategies.

2 Motivation and Problem Definition

2.1 Motivation

The MinGPT example in Figure 1 illustrates the different costs and opportunities of development at a fixed abstraction level. Figure 1(a) shows that High-only reaches a strong implementation early, whereas Low-only requires more search to reach higher performance. Continued high-level development remains limited to the controls exposed by the DSL. Low-level development exposes additional controls but requires the agent to manage more interacting implementation choices from the beginning. Exporting a promising high-level candidate allows low-level development to build on the progress already made. In Figure 1(b), the agent continues modifying the exported candidate and obtains further performance improvement. The exported code makes implementation details previously handled by the compiler available for direct modification. The agent can refine these details while retaining the computation structure developed during high-level search.

However, cross-level development requires deciding when to change abstraction levels and which candidate to continue from. A performance plateau may arise because the current representation cannot expose the control needed for further improvement, but the plateau can also appear before the agent finds effective modifications at the current level. Distinguishing between these possibilities requires examining the candidate’s code, performance feedback, and previous modification attempts together. The agent must identify useful modifications from the available evidence and select a source candidate and target abstraction level that support those modifications.

2.2 Problem Definition

We study how to guide continued kernel development across abstraction levels using existing candidates and evaluation feedback. The setting consists of a computational task, target hardware d , and a base agentic kernel workflow O . The base workflow generates, modifies, and evaluates candidate implementations. We focus on two editable abstraction levels, $\Lambda = \{H, L\}$, corresponding to high- and low-level representations, respectively.

Correctness checks and measured performance determine candidate quality. Each candidate c denotes a specific code version of a complete implementation of the task, which may contain multiple kernels. A candidate is expressed at one abstraction level $\ell(c) \in \Lambda$. Let $V(c) = 1$ indicate that c passes the task’s correctness checks. For a candidate that passes the checks, $m_d(c)$ denotes the execution time metric measured under the evaluation protocol for hardware d . Candidates that have not passed correctness checks may be retained for repair but do not enter performance rankings.

The refinement state records the candidates and feedback available at decision step t : $s_t = (c_t, C_t, \mathcal{H}_t)$. The current candidate is c_t , and C_t contains the candidates available for further editing, including c_t and retained historical versions. The history $\mathcal{H}_t$ records modifications, representation conversions, evaluation results, and available hardware evidence. Evaluation results and hardware evidence are associated with the corresponding candidate versions and evaluation conditions.

A development decision specifies an action, a source candidate, and a target abstraction level. Let $\mathcal{L}(c) \subseteq \Lambda$ denote the target levels available from candidate c . The set includes the current level $\ell(c)$ and lower levels for which conversion is supported. Returning to a higher level selects a retained high-level source. We seek a decision rule π of the following form where $c_t^{\text{src}} \in C_t$ and $\ell_t^{\text{tar}} \in \mathcal{L}(c_t^{\text{src}})$.

$$(a_t, c_t^{\text{src}}, \ell_t^{\text{tar}}) \leftarrow \pi(s_t) \quad (1)$$

The objective is to improve the performance of the final implementation under a given base workflow and execution protocol. The execution protocol specifies the run budget, acceptance requirements, and stopping conditions. The workflow returns the best measured correct candidate as c_{out} , satisfying $V(c_{\text{out}}) = 1$. Smaller values of $m_d(c_{\text{out}})$ indicate better performance.

3 TILEWEAVE

3.1 Overview

TILEWEAVE is a unified, pluggable method for feedback-guided programming of accelerator kernels across abstraction levels. As shown in Figure 2, TILEWEAVE integrates with the base workflow’s *Analyze & Plan* and *Evaluate & Record* stages. Integration requires access to candidate state and evaluation feedback and support for executing the selected plan. The base workflow retains its generation strategy and evaluation procedure.

The three components connect evidence checks, modification planning, and history updates within each iteration. The agent first checks whether code, feedback, and history support a concrete next modification. When evidence is insufficient, *Cross-Level Diagnosis* establishes source

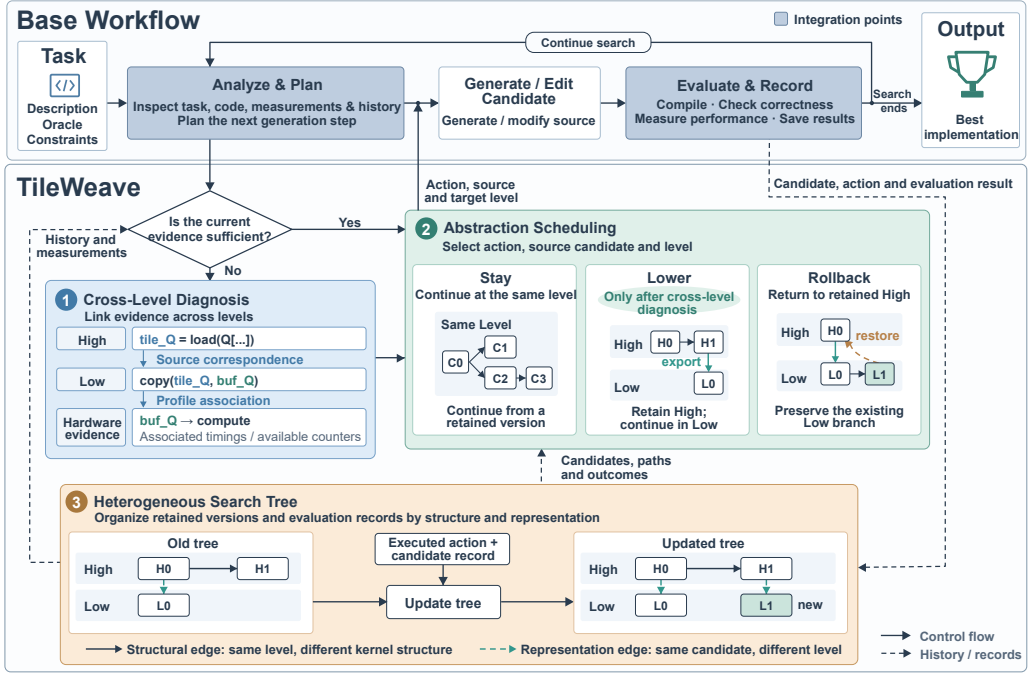


Fig. 2. Architecture and workflow of TILEWEAVE.

correspondence and associates hardware evidence with the executed code. *Abstraction Scheduling* uses evidence and history to compare controls and implementation requirements across levels and select an action, source candidate, and target level. After plan execution and evaluation, the *Heterogeneous Search Tree* records candidates, results, and relationships from modifications and conversions, preserving observation context for subsequent decisions.

We use MinGPT causal attention from Figure 1 as a running example, with TileLang at High and HIP C++ at Low on an AMD Radeon AI PRO R9700. Figures 3 and 4 illustrate diagnosis and candidate history.

Role of LLMs. The base workflow’s LLM-based agents follow TILEWEAVE’s instructions to interpret code and feedback, assess evidence sufficiency, propose modifications, and select actions, source candidates, and target levels. The coding model generates and revises candidates, and evaluation tools compile the code, check numerical correctness, and measure latency. Section 4.1 describes the evaluated models and experimental settings.

3.2 Cross-Level Diagnosis

Cross-Level Diagnosis helps the agent interpret execution feedback by examining a candidate’s implementation across abstraction levels. The agent first checks whether the available code, feedback, and history provide sufficient evidence for a concrete next modification. Evidence is sufficient when available code, feedback, or applicable earlier diagnostic findings allow the agent to justify a modification, identify the code to change, and state an expected outcome to test. When the evidence is insufficient, the agent invokes *Cross-Level Diagnosis* to establish the relevant connections through *Source correspondence* and *Profile association*.

(a) Source correspondence

iter-8

TileLang

ModelNew.py: 54

```
T.reduce_max(s, m, dim=1, clear=False)
```

Generated HIP C++

kernel_4.cpp:92–103

```
for (int i_8 = 0; i_8 < 8; ++i_8) {
    // Local maximum over this row ...
    __syncthreads();
    m_clear[i_8] = t1::AllReduce<t1::MaxOp,
        256, 128, 0>::run(m_clear[i_8],
        (&(((float*)workspace)[0])));
    m_clear[i_8] = t1::AllReduce<t1::MaxOp,
        16, 1, 0>::run(m_clear[i_8]);
    m[i_8] = max(m[i_8], m_clear[i_8]);
}
```

(b) Diagnosis

Code observations

Property	Observation
Memory	Shared memory
Synchronization	Block barrier
Operation	Max reduction

Measured performance

Whole attention call

2.786430 ms

The timing covers the entire attention call, not the reduction alone.

Fig. 3. *Cross-level diagnosis* of MinGPT at iter-8. (a) A TileLang reduction and a fragment of the associated generated HIP C++ implementation, shown as a code-inspection example. (b) Code observations and measured whole-attention-call latency for the same candidate.

Source correspondence connects a higher-level call to its generated lower-level implementation for a specific candidate version. During conversion, TILEWEAVE records the higher-level call site, the generated code, and how the call’s arguments are passed to the generated implementation. *Cross-Level Diagnosis* uses these records to identify the generated implementation associated with the call. In the running example, the component checks whether the call site, generated code, and argument bindings observed during execution match the conversion records. The agent inspects the associated code to identify the operations and data dependencies implementing the higher-level call.

Profile association links each measurement to the kernel or call group evaluated in a specific candidate. *Cross-Level Diagnosis* matches kernel identifiers or call relationships in the execution records to the corresponding kernel or call group in the candidate code. The association retains the candidate version and execution conditions recorded for the measurement. If the execution records do not identify a unique match, the measurement remains unassociated. The agent uses the measurement association to locate the measured code and follows source correspondence to inspect the related higher-level calls. For timing measurements, the recorded time covers the kernel or call group as a whole. Attributing part of the recorded time to an individual memory access or synchronization operation requires additional evidence.

To check whether earlier evidence remains applicable, *Cross-Level Diagnosis* compares the selected candidate’s code and argument bindings with the recorded source correspondence. For edited lower-level code, the recorded higher-level parent identifies the implementation’s origin but does not establish an exact correspondence. The agent checks whether the relevant code and execution conditions still support the earlier diagnostic findings.

In the running example, the two associations guide inspection of the attention implementation at restored High iter-8. Attention takes 2.786 ms, compared with 1.759 ms for QKV projection and 0.543 ms for activation conversion. The attention cost motivates inspection of the associated Low-level code. Figure 3 shows a High-level softmax reduction alongside a reduction fragment from the associated Low-level implementation. The call-level association identifies the generated implementation, while the operation-level comparison in the figure comes from code inspection.

Algorithm 1 *Abstraction Scheduling* in TILEWEAVE.

Require: Refinement state $s_t = (c_t, C_t, \mathcal{H}_t)$, task and hardware information τ
Ensure: Plan p , or $\perp$

$E \leftarrow \text{ExistingEvidence}(s_t, \tau)$
if not EvidenceSufficient(E) **then**
 $E \leftarrow \text{CrossLevelDiagnosis}(s_t, E)$
if not EvidenceSufficient(E) **then**
 $\text{return } \perp$
 $\mathcal{D} \leftarrow \text{ProposeChanges}(E)$
 $\mathcal{P} \leftarrow \text{FeasiblePlans}(\mathcal{D}, s_t, E, \tau)$
if $\mathcal{P} = \emptyset$ **then**
 $\text{return } \perp$
 $\text{return SelectPlan}(\mathcal{P}, E, \mathcal{H}_t)$

The fragment contains shared-memory exchanges and block synchronization, but the recorded time covers the whole attention call.

A separate observation in the same Low-level attention implementation motivates a hypothesis about the shared-memory layout of V . The generated code uses strided scalar reads for the PV operands, and vector reads for the QK operands. The difference in access patterns motivates testing whether changing the shared-memory layout of V improves operand access. Forming the hypothesis from Low-level code does not change the active abstraction level. Section 3.3 explains how the agent selects the implementation level for this hypothesis, completing example **A** in Figure 4.

3.3 Abstraction Scheduling

Abstraction Scheduling turns the available evidence into a plan for the next modification, including its source candidate and target abstraction level. The agent first reviews candidate code, evaluation feedback, and history, invoking *Cross-Level Diagnosis* when the evidence is insufficient under the criterion in Section 3.2. The agent then forms a testable modification hypothesis and identifies the controls needed to implement the change.

The agent identifies relevant source candidates from the current implementation and summaries of earlier modifications and evaluation results, and reads the relevant code and diagnostic records as needed. For each source under consideration, the agent checks whether its structure supports the modification and which levels expose the required controls. The supporting evidence must apply to that source and its execution conditions. When several choices are feasible, the agent compares their expected outcomes, implementation effort, and validation requirements. An earlier or slower candidate may be selected when its structure makes the modification easier to implement and evaluate.

Execution restores the selected source when necessary, and then follows one of three actions. *Lower* exports a supported lower-level representation and requires cross-level diagnostic evidence applicable to the source and proposed modification. *Rollback* restores a retained source whose abstraction level is higher than $\ell(c_t)$ and continues development there. *Stay* covers the remaining cases in which development continues in the source’s existing representation, including resuming a retained Low branch. Existing branches remain available after these actions.

The selected plan $p = (\Delta, a, c^{\text{src}}, \ell^{\text{tar}})$ specifies the modification, action, source candidate, and target level, together with a short rationale. The agent records the selected plan in the candidate history. Explicit enumeration or ranking of alternative plans is not required. After execution, the

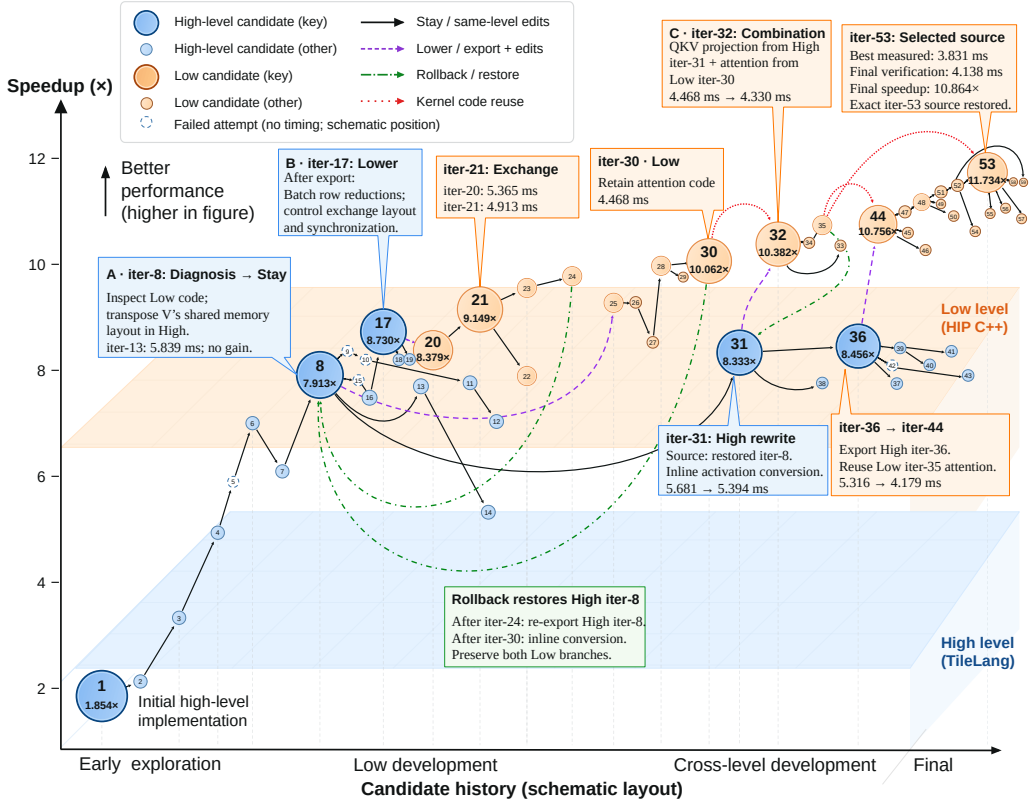


Fig. 4. MinGPT candidate history across High (blue) and Low (orange) development. Callouts A, B, and C highlight Stay, Lower, and cross-branch code reuse, respectively.

base workflow checks correctness and records the evaluation results in the *Heterogeneous Search Tree* for subsequent decisions. Only candidates that pass correctness checks enter performance comparisons. Algorithm 1 summarizes the agent’s planning steps. A return value of $\perp$ indicates that no supported plan is available and leaves the next step to the base workflow’s execution protocol.

For the *V*-layout hypothesis in example A of Figure 4, TileLang exposes the required shared-array shape, write indices, and matrix-operand orientation. The agent selects Stay from restored High iter-8, although the hypothesis arises from inspecting Low-level code. Evaluation confirms correctness but shows no performance improvement.

In example B of Figure 4, the agent selects Lower from the retained High candidate iter-17. Iter-17 remains the fastest correct High candidate after subsequent tile-size and *V*-layout variants fail to improve performance. After exploring several High structures, the agent selects Low for direct control over cache hints, loop unrolling, and load scheduling. Inspection of the exported code reveals serial row reductions and motivates batching to reduce repeated synchronization. The agent tests the change by directly modifying the reduction helpers, shared-memory exchange layout, and synchronization order. The first attempt is slower, while a subsequent revision improves on the High source.

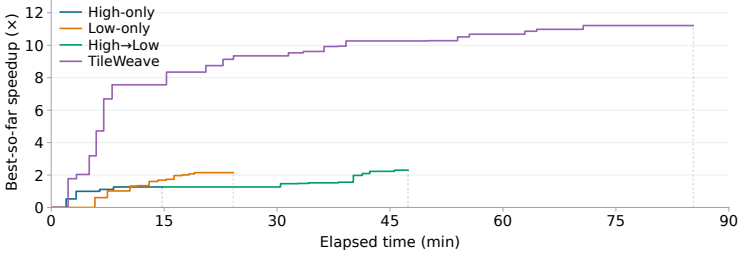


Fig. 5. Best-so-far speedup over elapsed time for MinGPT on an AMD Radeon AI PRO R9700. The comparison retains the High-only, Low-only, and High→Low trajectories from Figure 1(b) and adds the TILEWEAVE-guided trajectory.

3.4 Heterogeneous Search Tree

Heterogeneous Search Tree preserves candidate implementations and their development histories to support subsequent diagnosis and scheduling. Each candidate is retained as a complete, versioned implementation linked to an iteration record. The record identifies the candidate’s abstraction level, source parent, and associated evaluation and diagnostic records. Observations remain associated with the evaluated code version and execution conditions. The current candidate and the best correct candidate are tracked separately, allowing further exploration while retaining a candidate for final output.

Derivation relationships distinguish same-level modifications from representation conversions. A *Structural edge* records a structural modification at the same level, while a *Representation edge* records an actual conversion. Local parameter revisions retain their versions and source relationships within a branch. Restoring an earlier candidate preserves existing branches, and subsequent modifications create descendants of the restored source.

The agent uses the retained candidates and records to recover suitable starting points and assess whether earlier observations still apply. Evidence is reused after checking the relevant code and execution conditions. For cross-branch code reuse, the agent checks the computation and input/output interfaces and records the contributing candidate. The complete combined candidate undergoes the base workflow’s correctness checks and performance evaluation.

Example C in Figure 4 shows how retained candidates support cross-branch code reuse. Recorded activation conversion overhead and input-tile reuse in the grouped QKV projection motivate testing inline activation conversion. The agent restores High iter-8 to modify its grouped QKV projection, producing High iter-31. The agent selects iter-30, the best retained Low candidate, to reuse the attention improvements developed on that branch. After exporting iter-31, the agent checks that the attention computation and input/output interfaces remain compatible and reuses the code from iter-30. The combined candidate iter-32 passes correctness checks and runs in 4.330 ms, compared with 4.468 ms for iter-30.

Figure 5 extends the MinGPT comparison in Figure 1(b) by adding the TILEWEAVE-guided trajectory.

4 Evaluation

To evaluate TILEWEAVE, we conduct experiments addressing four research questions (RQs):

RQ1: End-to-End Effectiveness. How effective is TILEWEAVE in improving end-to-end kernel programming across platforms, coding models, and base workflows?

RQ2: Search Efficiency. How does TILEWEAVE compare with alternative abstraction-level strategies in terms of search efficiency?

RQ3: Component Contributions. How do Cross-Level Diagnosis, Abstraction Scheduling, and the Heterogeneous Search Tree respectively affect TILEWEAVE performance?

RQ4: Cross-Level Search Behavior. How effective are the cross-level scheduling decisions made by TILEWEAVE?

4.1 Experimental Setup

Workloads, platforms and models. We evaluate TILEWEAVE on 50 end-to-end workloads from KernelBench Level 3 [44]. Each workload’s reference implementation serves as both the correctness reference and performance baseline. We use the Ascend 910B3 [35] and AMD Radeon AI PRO R9700 [2], with TileLang [61] as the high-level representation and Ascend C [25] and HIP C++ [1] as the respective low-level representations. We use GPT-6 Astra [43] and DeepSeek-V4.1-Flash [13], yielding four accelerator–model configurations. Both models use low reasoning effort. Software details are available on the project website [56].

Base workflows and configurations. We use AKO [66] as the primary base workflow and evaluate five baselines: High, Low, Fixed, Unrestricted, and Random. High and Low restrict editing to the corresponding representation. Fixed performs an independent High-stage search, lowers its best validated candidate once, and starts a fresh Low-stage search without inherited context or history. Unrestricted retains AKO’s existing multi-representation support without the three TILEWEAVE components. Random uses the same legal cross-level actions as TILEWEAVE but chooses transitions through frequency-matched random routing. We compare these with AKO+TILEWEAVE.

We additionally compare KDA [42] with KDA+TILEWEAVE to evaluate reuse. Because KDA does not natively expose the High/Low interface, constructing representation-specific controls would require additional execution paths, conversion, and cross-stage handoff beyond its original workflow. We therefore compare the original KDA directly with KDA+TILEWEAVE. RQ1 uses full configuration names; RQ2–RQ4 use short names for AKO-based configurations.

Common evaluation protocol. Each method is evaluated on every workload–configuration pair using five independent agent seeds and the same evaluator. Each run has a 120-minute total budget including failed attempts, diagnosis, representation transitions, and candidate evaluation, although the base workflow may terminate earlier.

Correctness is checked against the KernelBench reference on the same generated inputs with tolerance 10^{-3} . Latency is the median of five timed executions after three warm-ups. A workload–configuration pair is successful if at least three of five runs produce a correct final implementation. Across pairs, the median and P90 coefficients of variation are 2.20% and 3.99%, respectively.

Metrics and aggregation. Section 2.2 defines the single-run objective using correctness $V(c)$ and absolute execution time $m_d(c)$: among correct candidates, lower $m_d(c_{\text{out}})$ is better. For cross-workload evaluation, we normalize the same latency quantity by the workload-specific reference. For a correct candidate c on hardware d , with c_{ref} denoting the corresponding KernelBench reference, we define

$$S(c) = \frac{m_d(c_{\text{ref}})}{m_d(c)}.$$

Thus, $S(c)$ is the speedup of an individual candidate, not an average. A seed-level run without a correct final implementation contributes zero. We first average endpoint scores across five seeds for each workload–configuration pair and then across workloads; the overall mean weights the four

accelerator–model configurations equally. Correctness, wall-clock time, and candidate evaluations separately characterize reliability and search cost.

We report standard deviation across five seed-level overall means and compute 95% confidence intervals using workload-level percentile bootstrap with paired observations preserved. Seed-level results and resampling details are available online [56].

For paired comparisons, if neither method meets majority correctness, the result is a tie; if only one does, it wins. Otherwise, let S_1 and S_2 be the five-seed mean endpoint speedups of the two methods on the same workload–configuration pair. We count a tie when

$$\frac{|S_1 - S_2|}{\max(S_1, S_2)} < 0.03,$$

and otherwise assign the win to the larger speedup.

For search-progress analyses, we average each run’s best correct speedup at each reporting budget across seeds and workloads. A run contributes zero until it becomes correct and carries forward its final value after termination.

4.2 RQ1: End-to-End Effectiveness

We compare AKO+TILEWEAVE and KDA+TILEWEAVE with the five AKO baselines and KDA on all 200 workload–configuration pairs. We measure endpoint speedup, majority-success correctness, and paired workload-level outcomes. Table 1 summarizes endpoint performance. AKO+TILEWEAVE achieves the highest mean speedup in every accelerator–model configuration: 4.576× on Ascend/Astra, 6.722× on AMD/Astra, 2.778× on Ascend/DeepSeek Flash, and 5.268× on AMD/DeepSeek Flash. Its equal-weight overall mean is 4.836×, compared with 3.748× for AKO-Random, the strongest AKO baseline by overall mean, corresponding to a 29.0% relative improvement. Among deterministic baselines, AKO-Fixed is strongest at 3.741×, which AKO+TILEWEAVE improves by 29.3%. The overall 95% intervals are [4.054, 5.704] for AKO+TILEWEAVE and [3.140, 4.434] for AKO-Random.

The gain is not driven by a single accelerator–model configuration. Relative to the strongest AKO baseline within each configuration, AKO+TILEWEAVE improves mean speedup by 22.0% on Ascend/Astra, 22.5% on AMD/Astra, 21.2% on Ascend/DeepSeek Flash, and 13.7% on AMD/DeepSeek Flash. Random is strongest on the two Astra configurations, while Fixed is strongest on the two DeepSeek Flash configurations.

Stability across agent runs. The endpoint results remain stable across the five independent agent seeds. At the aggregate level, the standard deviation is 0.221× for AKO+TILEWEAVE and 0.167× for AKO-Random, the strongest overall baseline. For the second base workflow, the corresponding values are 0.119× for KDA+TILEWEAVE and 0.044× for KDA. Together with the 2.20% median and 3.99% P90 coefficient of variation reported in the setup, these results indicate low sensitivity to agent stochasticity relative to the observed endpoint improvements.

Per-workload behavior. The arithmetic-mean improvements are also broad across workloads. Against AKO-Random, AKO+TILEWEAVE records 158 wins, 5 ties, and 37 losses under the 3% relative tie threshold, corresponding to a 79.0% win rate. Against AKO-Fixed, it records 144 wins, 6 ties, and 50 losses, corresponding to a 72.0% win rate. Against KDA, KDA+TILEWEAVE records 159 wins, 3 ties, and 38 losses, corresponding to a 79.5% win rate.

Reuse across base workflows. Relative to AKO-Unrestricted, adding TILEWEAVE increases the overall mean from 3.235× to 4.836×, a 49.5% improvement. For KDA, the same interface increases the overall mean from 2.615× to 4.425×, a 69.2% improvement. The KDA integration improves mean speedup in all four accelerator–model configurations.

Table 1. End-to-end endpoint performance. Overall mean is reported with standard deviation across five independent agent seeds; correctness uses majority success. Best values are bolded.

Method	Asc./Astra	AMD/Astra	Asc./DS	AMD/DS	Overall mean	Overall median	Correct
AKO-High	2.402	5.013	1.046	3.792	3.063±0.082	2.301	188/200
AKO-Low	3.032	3.153	1.981	2.915	2.770±0.085	2.208	192/200
AKO-Fixed	2.982	5.055	2.293	4.632	3.741±0.113	3.037	188/200
AKO-Unrestricted	2.444	4.696	1.856	3.942	3.235±0.123	2.473	192/200
AKO-Random	3.750	5.487	1.962	3.792	3.748±0.167	2.804	191/200
KDA	2.429	3.068	2.007	2.956	2.615±0.044	2.131	190/200
AKO+TILEWEAVE	4.576	6.722	2.778	5.268	4.836±0.221	3.767	195/200
KDA+TILEWEAVE	4.437	6.339	2.371	4.553	4.425±0.119	3.392	195/200

Table 2. Paired workload-level endpoint comparisons in RQ1. Each comparison contains the 200 workload-configuration pairs and uses the five-seed aggregated endpoint.

Comparison	Win / Tie / Loss	Win rate
AKO+TILEWEAVE vs. AKO-Random	158 / 5 / 37	79.0%
AKO+TILEWEAVE vs. AKO-Fixed	144 / 6 / 50	72.0%
KDA+TILEWEAVE vs. KDA	159 / 3 / 38	79.5%

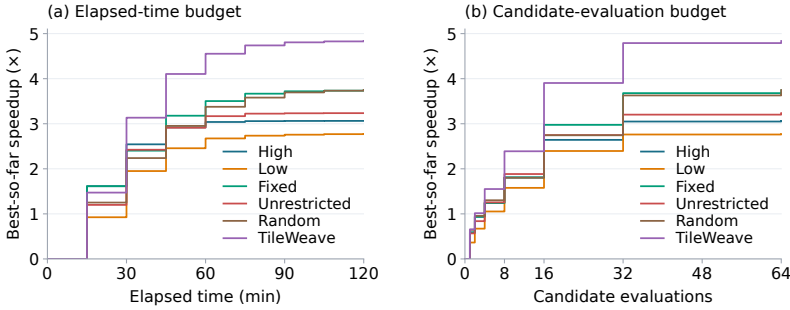


Fig. 6. Search progress. Best-so-far speedup is shown under (a) elapsed wall-clock time and (b) candidate-evaluation budget.

Discussion. AKO-Random is the strongest baseline by overall mean, although AKO-Fixed is within 0.2% of it. The strongest baseline also varies by configuration: Random is strongest on the two Astra configurations, while Fixed is strongest on the two DeepSeek Flash configurations. This indicates that no single abstraction-level strategy is universally best. Correctness failures are also more frequent under DeepSeek-V4.1-Flash than under Astra. Because failed seed-level runs contribute zero, this reliability difference is already reflected in endpoint speedup.

Key finding. AKO+TILEWEAVE achieves the highest mean speedup in all four accelerator-model configurations and improves the strongest AKO baseline by 29.0%. The gain is stable across five independent agent seeds and broad across individual workloads. The same interface improves KDA by 69.2%.

4.3 RQ2: Search Efficiency

We compare High, Low, Fixed, Unrestricted, Random, and TILEWEAVE under the same search budget. We measure both the cost of reaching the first correct implementation and best-so-far speedup as a function of elapsed wall-clock time and candidate evaluations. As shown in Table 3, High

Table 3. Cost of reaching the first correct implementation. Successful counts use majority success across five agent seeds.

Method	Successful	Time (min)↓	Evaluations↓
High	188/200	6.81	1.43
Low	192/200	12.17	2.06
Fixed	188/200	11.39	1.82
Unrestricted	192/200	11.96	1.94
Random	191/200	10.58	1.85
TILEWEAVE	195/200	10.74	1.80

reaches the first correct candidate fastest: 6.81 minutes and 1.43 candidate evaluations on average. Random requires 10.58 minutes and 1.85 evaluations, while TILEWEAVE requires 10.74 minutes and 1.80 evaluations and Low requires 12.17 minutes and 2.06 evaluations. Thus, TILEWEAVE’s endpoint advantage does not arise from producing the first correct implementation sooner.

The ordering changes during continued refinement. At 30 minutes, TILEWEAVE reaches 3.134 $\times$, compared with 2.543 $\times$ for High, 2.403 $\times$ for Fixed, and 2.238 $\times$ for Random. At 60 minutes, it reaches 4.553 $\times$, compared with 3.503 $\times$ for Fixed and 3.376 $\times$ for Random. By 90 minutes, TILEWEAVE reaches 4.806 $\times$, while Fixed and Random reach 3.720 $\times$ and 3.695 $\times$, respectively.

The same pattern remains when budget is measured by candidate evaluations. At 16 evaluations, TILEWEAVE reaches 3.904 $\times$, compared with 2.975 $\times$ for Fixed, 2.744 $\times$ for Random, and 2.642 $\times$ for High. At 32 evaluations, TILEWEAVE reaches 4.790 $\times$, compared with 3.679 $\times$ for Fixed, 3.628 $\times$ for Random, and 3.049 $\times$ for High.

Search saturation. The trajectories exhibit clear saturation near the end of the search budget. For TILEWEAVE, mean best-so-far speedup increases only from 4.806 $\times$ at 90 minutes to 4.836 $\times$ at 120 minutes, an additional improvement of approximately 0.6%. Likewise, doubling the candidate-evaluation budget from 32 to 64 increases mean speedup only from 4.790 $\times$ to 4.836 $\times$, an additional improvement of approximately 1.0%. The baselines show the same qualitative flattening, indicating that the workflows approach search saturation within the allocated budget.

Discussion. The faster first correctness of High does not imply more effective search. A key feature of TILEWEAVE is that exploratory failures are recoverable: Cross-Level Diagnosis turns unsuccessful attempts into actionable feedback, the Heterogeneous Search Tree preserves validated states and alternative branches, and Abstraction Scheduling can redirect development when an attempt does not succeed. Together, these mechanisms allow less conservative exploration without requiring every intermediate candidate to be immediately correct.

An Ascend/Astra MLP run illustrates this behavior. The High-stage baseline reaches correctness after 4.13 minutes, whereas TILEWEAVE requires 5.41 minutes because its first, more aggressive High-level candidate fails compilation and is repaired in the next iteration. Both attempts occur before any Lower or Rollback, so the delay is not caused by cross-level transition overhead. The complete trace is available online [56].

The advantage of TILEWEAVE also persists when progress is measured by candidate evaluations rather than elapsed time, making a purely per-iteration runtime explanation unlikely. The saturation under both budget views further shows that TILEWEAVE reaches a stronger performance level rather than merely postponing improvements to the end of a longer search.

Table 4. Ablation results. Mean speedup is reported with standard deviation across five agent seeds. Correctness uses majority success. Loss is relative to the complete TILEWEAVE configuration.

Variant	Correct (%)↑	Mean speedup↑	Evaluations↓	Wall time (min)↓	Loss
Full TILEWEAVE	97.5	4.836±0.221	21.76	52.05	–
w/o Diagnosis	96.5	4.229±0.032	22.82	50.14	12.6%
w/o Scheduling	94.5	3.854±0.147	22.47	53.04	20.3%
w/o Tree	96.5	4.141±0.041	22.20	54.71	14.4%

Key finding. High reaches correctness sooner, but TILEWEAVE uses the subsequent search budget more effectively. Its advantage persists under both wall-clock and candidate-evaluation budgets and reaches a substantially stronger saturation level.

4.4 RQ3: Component Contributions

We perform one-at-a-time ablations of the three TILEWEAVE components while keeping the common workload, model, evaluator, seed, and search-budget protocol unchanged. w/o Diagnosis removes Cross-Level Diagnosis. w/o Scheduling replaces evidence-guided Abstraction Scheduling with the fixed High-to-Low progression. w/o Tree removes editable access to alternative historical branches while retaining the incumbent.

Results. Table 4 shows that every ablation reduces endpoint performance. Removing Diagnosis lowers mean speedup from 4.836× to 4.229×, a 12.6% reduction. Removing Scheduling lowers it to 3.854×, the largest reduction at 20.3%. Removing the Heterogeneous Search Tree lowers it to 4.141×, a 14.4% reduction.

The variants consume similar search budgets. The complete configuration averages 21.76 candidate evaluations and 52.05 minutes, compared with 22.82/50.14, 22.47/53.04, and 22.20/54.71 for the three ablations. The endpoint differences therefore cannot be explained by the complete configuration simply evaluating substantially more candidates.

Discussion. The ablation losses are not additive. Diagnosis changes the evidence available to Scheduling, Scheduling changes which branches are explored, and the Heterogeneous Search Tree changes which historical states remain available to both mechanisms. The slightly shorter wall time of w/o Diagnosis is therefore not evidence that Diagnosis is unnecessary: removing diagnostic work reduces overhead but also produces a substantially weaker endpoint.

Key finding. Removing Diagnosis, Scheduling, or the Heterogeneous Search Tree reduces mean speedup by 12.6%, 20.3%, and 14.4%, respectively, under comparable search budgets.

4.5 RQ4: Cross-Level Search Behavior

RQ4 combines whole-run routing controls, post-action analysis, and matched interventions to distinguish the value of cross-level movement from the value of selecting when to move.

Frequency-matched random control. For each accelerator–model configuration, we derive the empirical conditional transition rates from the corresponding TILEWEAVE trajectories:

$$p_L = \frac{N_{\text{Lower}}}{N_{\text{High boundaries}}}, \quad p_R = \frac{N_{\text{Rollback}}}{N_{\text{Low boundaries}}}.$$

At a High boundary, Random selects Lower with probability p_L and Stay otherwise. At a Low boundary, Random selects Rollback with probability p_R and Stay otherwise. The frequency-matched Random policy uses aggregate conditional probabilities of 14.22% for Lower given High

Table 5. Whole-run routing-policy comparison. Mean speedup is reported with standard deviation across five independent agent seeds.

Policy	Mean speedup $\uparrow$	Evals./run $\downarrow$	Time (min) $\downarrow$	Lower/run	Rollback/run
Fixed	3.741 $\pm$ 0.113	23.94	54.34	0.935	0.000
Random	3.748 $\pm$ 0.167	23.28	54.71	1.368	0.723
Unrestricted	3.235 $\pm$ 0.123	18.34	40.17	–	–
TILEWEAVE	4.836$\pm$0.221	21.76	52.05	1.255	0.540

Table 6. Performance following routing actions in TILEWEAVE trajectories. Productive denotes an improvement of more than 3% within the next three candidate evaluations.

Action	Productive $\uparrow$	Median gain $\uparrow$
Stay	83.4%	13.12%
Lower	84.4%	27.37%
Rollback	64.4%	6.99%

and 5.88% for Rollback given Low. Random otherwise follows the common evaluation protocol and ignores diagnostic evidence when choosing when to transition.

Whole-run routing results. Table 5 compares the complete routing policies. Random reaches 3.748 $\times$ mean speedup, whereas TILEWEAVE reaches 4.836 $\times$, a 29.0% relative improvement. The difference holds in all four accelerator–model configurations: 22.0% on Ascend/Astra, 22.5% on AMD/Astra, 41.6% on Ascend/DeepSeek Flash, and 38.9% on AMD/DeepSeek Flash.

Random performs more cross-level movements per run than TILEWEAVE: 1.368 versus 1.255 Lower actions and 0.723 versus 0.540 Rollbacks. Its weaker endpoint therefore cannot be explained by receiving fewer opportunities to move between abstraction levels.

Post-action outcomes. A short window is productive if its best correct candidate improves the pre-action incumbent by more than 3% within the next three candidate evaluations.

Lower has the strongest immediate effect: 84.4% of eligible windows are productive, with a median gain of 27.37%. Rollback is weaker in the immediate three-evaluation window. When the observation window is extended from a Rollback until the next Rollback or run termination, however, 79.8% of eligible windows are productive, with a median gain of 50.07%. Because this longer window contains subsequent modifications, the latter statistic is descriptive rather than an isolated causal effect of Rollback.

Matched interventions. To separate action choice from the state at which the action occurs, we sample 25 eligible High switching checkpoints and 25 eligible Low switching checkpoints. At High checkpoints, Lower is compared with Stay; at Low checkpoints, Rollback is compared with Stay. Both continuations begin from the same candidate state and receive the same model, evaluator, and local budget of 10 candidate evaluations.

For a continuation, we define

$$g = 100 \left(\frac{S_{\text{after}}}{S_{\text{before}}} - 1 \right),$$

where S_{before} is the checkpoint speedup and S_{after} is the best correct speedup obtained within the next 10 candidate evaluations.

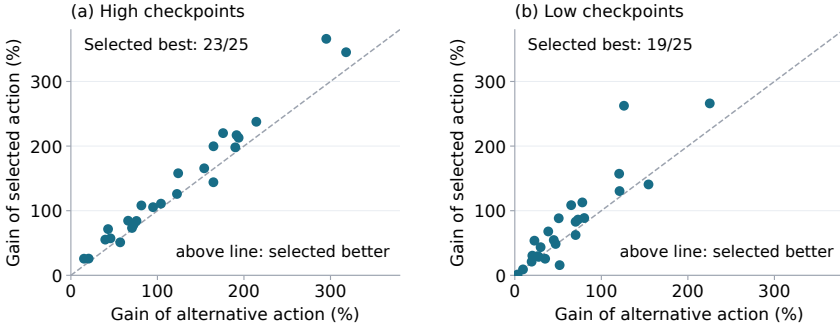


Fig. 7. Matched-checkpoint interventions in the TILEWEAVE workflow. Each point compares the selected cross-level continuation with Stay from the same candidate state under the same 10-evaluation local budget. Points above the diagonal favor the selected cross-level action.

At High checkpoints, Lower achieves the better tested continuation in 23 of 25 cases (92.0%). At Low checkpoints, Rollback is better in 19 of 25 cases (76.0%). Across all 50 interventions, the selected cross-level action is better in 42 cases (84.0%), with mean regret of 1.92 percentage points and median regret of zero.

Discussion. The whole-run controls progressively eliminate simpler explanations for the end-point improvement. Fixed shows that a predetermined one-way High-to-Low progression is insufficient. Random shows that merely making cross-level transitions at similar frequencies is also insufficient. The matched interventions instead show that the current search state matters: the TILEWEAVE-selected cross-level continuation is the better tested choice in 84.0% of sampled switching checkpoints.

Lower has a strong immediate effect, whereas Rollback often has delayed value. Rollback can leave a low-value local direction and reopen a retained High branch from which later Low candidates follow a different development path. The complete traces are available online [56].

The conditional probabilities used by Random are estimated from the same evaluation cohort. Random should therefore be interpreted as a post-hoc transition-frequency control rather than an independently trained or out-of-sample routing policy.

Key finding. Cross-level movement is useful because TILEWEAVE selects when and where to move, not merely because transitions are available. TILEWEAVE outperforms frequency-matched random routing by 29.0%, and its selected cross-level action is the better tested continuation in 84.0% of matched checkpoints.

5 Related Work

Agentic kernel generation and optimization. AKO [66] provides autonomous coding agents with benchmarking, profiling, trajectory recording, and language switching; KDA [42] structures kernel development around workload contracts, candidate generation, correctness checks, and performance evaluation. TILEWEAVE extends these workflows and inherits their optimization capabilities. Our integrations preserve this development-and-evaluation loop; selecting the programming level is an additional decision made using the available evidence and search history. Related systems strengthen planning, profiling, and reuse of historical candidates across platforms [9, 14, 15, 18, 36, 51, 67, 70, 72], while execution-based rewards improve trained kernel generators [32, 33, 54]. EGG [22] separates algorithmic design from hardware tuning, and HIERA [60] selects among

framework operators, libraries, and custom kernels. TILEWEAVE jointly selects a retained candidate and editable level, supporting continued edits, lowering, and revisiting higher-level branches. Existing benchmarks broaden workload, platform, generalization, and verification coverage [29, 41, 44, 63, 67, 69]; we use KernelBench Level 3 [44] as the common workload contract.

Agent–compiler cooperation and agent-oriented abstractions. ACCLAIM [39] optimizes C programs on CPUs using correctness and runtime feedback to coordinate LLM rewrites with unchanged compiler components. The reported feedback does not explicitly associate runtime costs with corresponding code across abstraction levels. Under an end-to-end model compilation view, TILEWEAVE augments base agentic workflows as backend components for kernel optimization through cross-level diagnosis. This diagnosis links measured kernel costs to DSL operations and generated Low-level implementations, helping the agent determine whether a proposed optimization requires DSL-level changes or direct Low-level edits. Ave [38] (formerly ARGUS) checks data-flow invariants in a tile-based DSL and returns localized counterexamples, supporting targeted repair while hiding lower-level representations. CAKE [68] exposes warp roles, memory movement, synchronization, and pipelines through a typed IR with verification and localized diagnostics; its IR and compiler evolve with kernel requirements. Ave and CAKE improve the controls and feedback within their designed abstractions; TILEWEAVE asks when development should stay at one level or resume elsewhere. Related approaches [10, 11, 19, 23, 53, 55] improve agent-facing abstractions, source-level edits, compiler representations, and transformation search. Optimization environments and learned compiler policies expose further decision spaces [12, 21, 34, 58], while execution feedback and self-refinement improve program selection and repair [5, 8, 20, 31, 37, 40, 52]. TILEWEAVE builds on these ideas through cross-representation evidence association and joint candidate/level selection, preserving alternative branches across existing High and Low representations without requiring a new IR.

Programming abstractions and multi-level search. Feedback-guided implementation search pre-dates LLMs: SPIRAL [47] optimizes SPL formulas and generated C/Fortran code, while ATLAS [64] and FFTW [17] adapt numerical implementations to hardware. Structured languages, compiler infrastructures, and user-extensible schedules expose complementary control over computation and implementation [6, 26–28, 30, 49, 57, 59, 61]. Autotuning and equality saturation explore or retain implementation alternatives [4, 7, 65, 71]; STOKE [50], BOLT [45], and CuAsmRL [24] continue optimization after compilation. Agent pipelines also combine DSL-guided lowering or semantic refactoring with subsequent optimization [48, 62]. Collectively, these lines of work show the value of structured representations, staged refinement, and search over alternative implementations. TILEWEAVE studies how runtime evidence and candidate history guide a coding agent’s next programming context: which implementation to resume, which level exposes the needed controls, and which cross-level branches to retain for later continuation.

6 Limitations and Threats to Validity

Evaluation scope. Our evaluation covers two editable abstraction levels, 50 KernelBench Level 3 workloads, two accelerator platforms, and two coding models. Generalization to additional abstraction levels and compiler representations remains to be evaluated. A compiler update may expose previously unavailable controls at the high level, changing which level supports a proposed modification. The selected workloads also limit conclusions about dynamic shapes, communication, and multi-device execution.

Diagnosis and scheduling. The quality of diagnosis and scheduling depends on the available evidence and the coding model’s interpretation. Kernel or call-group timings identify expensive

regions but provide limited evidence about individual memory accesses or synchronization operations. An incorrect interpretation can lead to ineffective modifications or unnecessary transitions, consuming the search budget. Edits and conversions can change the relevance of earlier observations despite preserved candidate histories.

Measurement and variability. Correctness and performance results depend on the tested inputs and run-to-run variability. The output checks use generated inputs and a numerical tolerance of 10^{-3} , leaving behavior on other inputs unverified. Warm-up executions and median timing reduce measurement noise, but device state may still affect measured latency. Model sampling also changes search trajectories. We quantify this variation using five independent agent seeds and report across-seed variability, while workload-level bootstrap intervals capture uncertainty across workloads.

Intervention scope. The matched interventions compare the selected cross-level action against Stay at 50 sampled switching checkpoints under the same 10-evaluation budget. The results assess local continuation choices and do not evaluate whether Stay decisions at other checkpoints are appropriate. Longer continuations or different modifications may change which action performs best. In the separate trajectory analysis, longer Rollback observation windows capture the combined effects of candidate restoration and subsequent edits, so the observed gains cannot be attributed to restoration alone.

Portability. Reuse across base workflows has been evaluated through the AKO and KDA integrations. Additional workflows require interfaces for exposing candidate state and evaluation feedback, executing selected plans, and restoring retained candidates. New backends also require support for code conversion and source correspondence, and profiler availability may limit the evidence available for diagnosis. The experiments assess kernel performance after integration and do not quantify the engineering effort required for these adaptations.

7 Conclusion

Abstraction levels shape the search space available to coding agents during accelerator-kernel development. High-level representations simplify structural exploration, while native code exposes finer control over hardware operations. Restricting development to one level can limit progress, but using both requires deciding what to modify, where to make the change, and which candidate to continue from.

In this paper, we have presented TILEWEAVE to address these decisions through Cross-Level Diagnosis, Abstraction Scheduling, and a Heterogeneous Search Tree. These components relate execution feedback to candidate code, guide the next modification and its implementation level, and preserve candidate histories for subsequent decisions. Evaluation with AKO and KDA demonstrates the benefits of feedback-guided cross-level programming across different base workflows. The experiments cover 50 KernelBench Level 3 workloads across two accelerator platforms and two language models. AKO with TILEWEAVE improves overall arithmetic mean speedup by 29.0% over the strongest evaluated baseline. Integrating TILEWEAVE with KDA improves the same metric by 69.2% over KDA. These results support treating abstraction selection as an explicit part of agent-driven kernel development, guided by execution evidence and the history of prior exploration.

Data Availability

The source code, additional experimental details and results, and supplementary materials are available on the project website [56].

References

- [1] Advanced Micro Devices, Inc. [n. d.]. HIP Documentation. <https://rocm.docs.amd.com/projects/HIP/en/latest/index.html> Accessed: 2026-09-04.
- [2] Advanced Micro Devices, Inc. 2025. AMD Radeon AI PRO R9700. <https://www.amd.com/en/products/graphics/workstations/radeon-ai-pro-ai-9000-series/amd-radeon-ai-pro-r9700.html> Official product page, accessed 2026-10-03.
- [3] Advanced Micro Devices, Inc. 2025. Introducing AMD CDNA 3 Architecture. Architecture white paper. <https://www.amd.com/content/dam/amd/en/documents/instinct-tech-docs/white-papers/amd-cdna-3-white-paper.pdf>
- [4] Jason Ansel, Shoaib Kamil, Kalyan Veeramachaneni, Jonathan Ragan-Kelley, Jeffrey Bosboom, Una-May O'Reilly, and Saman Amarasinghe. 2014. OpenTuner: An Extensible Framework for Program Autotuning. In *Proceedings of the International Conference on Parallel Architectures and Compilation Techniques*. <https://opentuner.org/publications/>
- [5] Bei Chen, Fengji Zhang, Anh Nguyen, Daoguang Zan, Zeqi Lin, Jian-Guang Lou, and Weizhu Chen. 2023. CodeT: Code Generation with Generated Tests. In *The Eleventh International Conference on Learning Representations*. <https://openreview.net/forum?id=ktw68Cmu9c>
- [6] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. 2018. TVM: An Automated End-to-End Optimizing Compiler for Deep Learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. USENIX Association, 578–594. <https://www.usenix.org/conference/osdi18/presentation/chen>
- [7] Tianqi Chen, Lianmin Zheng, Eddie Yan, Ziheng Jiang, Thierry Moreau, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. 2018. Learning to Optimize Tensor Programs. In *Advances in Neural Information Processing Systems*, Vol. 31. Curran Associates, Inc. <https://proceedings.neurips.cc/paper/2018/hash/8b5700012be65c9da25f49408d959ca0-Abstract.html>
- [8] Xinyun Chen, Maxwell Lin, Nathanael Schaerli, and Denny Zhou. 2024. Teaching Large Language Models to Self-Debug. In *International Conference on Learning Representations*. 8746–8825. https://proceedings.iclr.cc/paper_files/paper/2024/hash/2460396f2d0d421885997dd1612ac56b-Abstract-Conference.html
- [9] Kaiming Cheng, Laura Wang, Jack Khuu, Mark Saroufim, Wenyuan Chi, Jiannan Wang, and Joe Isaacson. 2026. KernelAgent: Hardware-Guided GPU Kernel Optimization via Multi-Agent Orchestration. PyTorch Blog. <https://pytorch.org/blog/kernelagent-hardware-guided-gpu-kernel-optimization-via-multi-agent-orchestration/>
- [10] Chris Cummins, Volker Seeker, Dejan Grubisic, Mostafa Elhoushi, Youwei Liang, Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Kim Hazelwood, Gabriel Synnaeve, and Hugh Leather. 2023. Large Language Models for Compiler Optimization. arXiv:2309.07062 <https://arxiv.org/abs/2309.07062>
- [11] Chris Cummins, Volker Seeker, Dejan Grubisic, Baptiste Roziere, Jonas Gehring, Gabriel Synnaeve, and Hugh Leather. 2025. LLM Compiler: Foundation Language Models for Compiler Optimization. In *Proceedings of the 34th ACM SIGPLAN International Conference on Compiler Construction*. ACM, 141–153. doi:10.1145/3708493.3712691
- [12] Chris Cummins, Bram Wasti, Jiadong Guo, Brandon Cui, Jason Ansel, Sahir Gomez, Somya Jain, Jia Liu, Olivier Teytaud, Benoit Steiner, Yuandong Tian, and Hugh Leather. 2022. CompilerGym: Robust, Performant Compiler Optimization Environments for AI Research. In *2022 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. IEEE, 92–105. doi:10.1109/cgo53902.2022.9741258
- [13] DeepSeek-AI. 2026. DeepSeek-V4.1-Flash: Pushing the Limits of KV Cache Compression. <https://huggingface.co/deepseek-ai/DeepSeek-V4.1-Flash> Accessed: 2026-09-23.
- [14] Juncheng Dong, Yang Yang, Tao Liu, Yang Wang, Feng Qi, Vahid Tarokh, Kaushik Rangadurai, and Shuang Yang. 2026. STARK: Strategic Team of Agents for Refining Kernels. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=nWaZTH1JMx>
- [15] Jinye Du, Quan Yuan, Zuyao Zhang, Yanzhi Yi, Jiahui Hu, Wangyi Chen, Yiyang Zhu, Qishui Zheng, Wenxiang Zou, Xiangyu Chang, Zuohe Zheng, Zichun Ye, Chao Liu, Shanni Li, Renwei Zhang, Yiping Deng, Xinwei Hu, Xuefeng Jin, and Jie Zhao. 2025. AKG Kernel Agent: A Multi-Agent Framework for Cross-Platform Kernel Synthesis. arXiv:2512.23424 <https://arxiv.org/abs/2512.23424>
- [16] Marina Favaro and Jack Clark. 2026. When AI Builds Itself. <https://www.anthropic.com/institute/recursive-self-improvement>. The Anthropic Institute.
- [17] Matteo Frigo and Steven G. Johnson. 2005. The Design and Implementation of FFTW3. *Proc. IEEE* 93, 2 (2005), 216–231. doi:10.1109/JPROC.2004.840301
- [18] Jiading Gai, Shuai Zhang, Kaj Bostrom, Jin Huang, Vihang Patil, Haoyang Fang, Bernie Wang, Huzefa Rangwala, and George Karypis. 2026. Optimizing CUDA like a Human: Micro-Profiling Tools as Expert Surrogates for LLM-Based GPU Kernel Optimization. arXiv:2606.26453 <https://arxiv.org/abs/2606.26453>
- [19] Ruijie Gao, Jirong Yang, Barry Lyu, Haoran Jin, and Nathan Bleier. 2026. Rethinking Agentic Kernel Generation for Emerging Accelerators. arXiv:2608.00894 <https://arxiv.org/abs/2608.00894>

- [20] Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Nan Duan, and Weizhu Chen. 2024. CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing. In *International Conference on Learning Representations*. 57734–57811. https://proceedings.iclr.cc/paper_files/paper/2024/hash/fe126561bbf9d4467dbb8d27334b8fe-Abstract-Conference.html
- [21] Ameer Haj-Ali, Qijing (Jenny) Huang, John Xiang, William Moses, Krste Asanovic, John Wawrzynek, and Ion Stoica. 2020. AutoPhase: Juggling HLS Phase Orderings in Random Forests with Deep Reinforcement Learning. In *Proceedings of Machine Learning and Systems*, Vol. 2. 70–81. https://proceedings.mlsys.org/paper_files/paper/2020/hash/5b47430e24a5a1f9fe21f0e8eb814131-Abstract.html
- [22] Yaochen Han, Ke Fan, Hongxu Jiang, Wanqi Xu, Weiye Xie, Runhua Zhang, Chenhui Zhu, and Yixiang Zhang. 2026. EGG: An Expert-Guided Agent Framework for Kernel Generation. In *Proceedings of the 43rd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 306)*. PMLR, 39691–39720. <https://proceedings.mlr.press/v306/han26h.html>
- [23] Siva Kumar Sastry Hari, Vignesh Balaji, Sana Damani, Qijing Huang, and Christos Kozyrakis. 2026. Improving Efficiency of GPU Kernel Optimization Agents using a Domain-Specific Language and Speed-of-Light Guidance. arXiv:2603.29010 <https://arxiv.org/abs/2603.29010>
- [24] Guoliang He and Eiko Yoneki. 2025. CuAsmRL: Optimizing GPU SASS Schedules via Deep Reinforcement Learning. In *Proceedings of the 23rd ACM/IEEE International Symposium on Code Generation and Optimization*. ACM, 493–506. doi:10.1145/3696443.3708943
- [25] Huawei Technologies Co., Ltd. [n.d.]. Ascend C. <https://www.hiascend.com/cann/ascend-c> Accessed: 2026-09-04.
- [26] Yuka Ikarashi, Gilbert Louis Bernstein, Alex Reinking, Hasan Genc, and Jonathan Ragan-Kelley. 2022. Exocompilation for Productive Programming of Hardware Accelerators. In *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation*. 703–718. doi:10.1145/3519939.3523446
- [27] Yuka Ikarashi, Kevin Qian, Samir Droubi, Alex Reinking, Gilbert Bernstein, and Jonathan Ragan-Kelley. 2025. Exo 2: Growing a Scheduling Language. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*. ACM, 426–444. doi:10.1145/3669940.3707218
- [28] Fredrik Kjolstad, Shoaib Kamil, Stephen Chou, David Lugato, and Saman Amarasinghe. 2017. The Tensor Algebra Compiler. *Proceedings of the ACM on Programming Languages* 1, OOPSLA (2017), 77:1–77:29. doi:10.1145/3133901
- [29] Robert Tjarko Lange, Qi Sun, Aaditya Prasad, Maxence Faldor, Yujin Tang, and David Ha. 2025. Towards Robust Agentic CUDA Kernel Benchmarking, Verification, and Optimization. arXiv:2509.14279 <https://arxiv.org/abs/2509.14279>
- [30] Chris Lattner, Mehdi Amini, Uday Bondhugula, Albert Cohen, Andy Davis, Jacques Pienaar, River Riddle, Tatiana Shpeisman, Nicolas Vasilache, and Oleksandr Zinenko. 2021. MLIR: Scaling Compiler Infrastructure for Domain Specific Computation. In *2021 IEEE/ACM International Symposium on Code Generation and Optimization*. 2–14. doi:10.1109/CGO51591.2021.9370308
- [31] Hung Le, Yue Wang, Akhilesh Deepak Gotmare, Silvio Savarese, and Steven Chu Hong Hoi. 2022. CodeRL: Mastering Code Generation through Pretrained Models and Deep Reinforcement Learning. In *Advances in Neural Information Processing Systems*, Vol. 35. 21314–21328. doi:10.52202/068431-1549
- [32] Shangzhan Li, Zefan Wang, Ye He, Yuxuan Li, Qi Shi, Jianling Li, Yonggang Hu, Wanxiang Che, Xu Han, Zhiyuan Liu, and Maosong Sun. 2025. AutoTriton: Automatic Triton Programming with Reinforcement Learning in LLMs. arXiv:2507.05687 <https://arxiv.org/abs/2507.05687>
- [33] Xiaoya Li, Xiaofei Sun, Albert Wang, Jiwei Li, and Chris Shum. 2026. CUDA-L1: Improving CUDA Optimization via Contrastive Reinforcement Learning. In *International Conference on Learning Representations*. 123922–123957. https://proceedings.iclr.cc/paper_files/paper/2026/hash/c94bbbf466ab1b2cfa100e41413b3a8-Abstract-Conference.html
- [34] Youwei Liang, Kevin Stone, Ali Shameli, Chris Cummins, Mostafa Elhoushi, Jiadong Guo, Benoit Steiner, Xiaomeng Yang, Pengtao Xie, Hugh James Leather, and Yuandong Tian. 2023. Learning Compiler Pass Orders using Coreset and Normalized Value Prediction. In *Proceedings of the 40th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 202)*. PMLR, 20746–20762. <https://proceedings.mlr.press/v202/liang23f.html>
- [35] Heng Liao, Jiajin Tu, Jing Xia, Hu Liu, Xiping Zhou, Honghui Yuan, and Yuxing Hu. 2021. Ascend: a Scalable and Unified Architecture for Ubiquitous Deep Neural Network Computing: Industry Track Paper. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 789–801. doi:10.1109/HPCA51647.2021.00071
- [36] Mao Luo, Hongbin Li, Feng Lin, Hanling Yi, and Zhe Huang. 2026. Agentic Kernel Optimization: Generating State-of-the-Art GPU Kernels Without Hand-Written CUDA. arXiv:2608.14560 <https://arxiv.org/abs/2608.14560>
- [37] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems*, Vol. 36. 46534–46594. doi:10.52202/075280-2019

- [38] Haohui Mai, Xiaoyan Guo, Xiangyun Ding, Daifeng Li, Qiuchu Yu, Chenzhun Guo, Cong Wang, Jiacheng Zhao, Christos Kozyrakis, and Binhang Yuan. 2026. Ave: Guiding Agentic GPU Optimization Using Data-Flow Invariants. In *Proceedings of the ACM SIGOPS 32nd Symposium on Operating Systems Principles*. ACM, 1055–1069. doi:10.1145/3830418.3843902
- [39] Benjamin Mikek, Danylo Vashchilenko, Bryan Lu, and Panpan Xu. 2026. Agentic Code Optimization via Compiler–LLM Cooperation. arXiv:2604.04238 <https://arxiv.org/abs/2604.04238>
- [40] Ansong Ni, Srini Iyer, Dragomir Radev, Veselin Stoyanov, Wen-Tau Yih, Sida Wang, and Xi Victoria Lin. 2023. LEVER: Learning to Verify Language-to-Code Generation with Execution. In *Proceedings of the 40th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 202)*. PMLR, 26106–26128. <https://proceedings.mlr.press/v202/ni23b.html>
- [41] Jiayi Nie, Haoran Wu, Yao Lai, Zeyu Cao, Cheng Zhang, Binglei Lou, Erwei Wang, Jianyi Cheng, Timothy M. Jones, Robert D. Mullins, Rika Antonova, and Yiren Zhao. 2026. KernelCraft: Benchmarking for Agentic Close-to-Metal Kernel Generation on Emerging Hardware. In *Proceedings of the 43rd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 306)*. PMLR, 93070–93121. <https://proceedings.mlr.press/v306/nie26c.html>
- [42] NVIDIA Research. 2026. Kernel Design Agents. <https://nvlabs.github.io/kda/>. Project page.
- [43] OpenAI. 2026. GPT-6 Astra. OpenAI API Documentation. <https://developers.openai.com/api/docs/models/gpt-6-astra> Accessed: 2026-09-23.
- [44] Anne Ouyang, Simon Guo, Simran Arora, Alex L. Zhang, William Hu, Christopher Re, and Azalia Mirhoseini. 2025. KernelBench: Can LLMs Write Efficient GPU Kernels?. In *Proceedings of the 42nd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 267)*. PMLR, 47356–47415. <https://proceedings.mlr.press/v267/ouyang25a.html>
- [45] Maksim Panchenko, Rafael Auler, Bill Nell, and Guilherme Ottoni. 2019. BOLT: A Practical Binary Optimizer for Data Centers and Beyond. In *2019 IEEE/ACM International Symposium on Code Generation and Optimization*. doi:10.1109/CGO.2019.8661201
- [46] Sundar Pichai. 2026. Cloud Next '26: Momentum and Innovation at Google Scale. <https://blog.google/innovation-and-ai/infrastructure-and-cloud/google-cloud/cloud-next-2026-sundar-pichai/>. Google, April 22, 2026.
- [47] Markus Püschel, José M. F. Moura, Jeremy Johnson, David Padua, Manuela Veloso, Bryan W. Singer, Jianxin Xiong, Franz Franchetti, Aca Gajić, Yevgen Voronenko, Kang Chen, Robert W. Johnson, and Nicholas Rizzolo. 2005. SPIRAL: Code Generation for DSP Transforms. *Proc. IEEE* 93, 2 (2005), 232–275. doi:10.1109/JPROC.2004.840306
- [48] Qiuyi Qu, Yicheng Sui, Yufei Sun, Rui Chen, Xiaofei Zhang, Yuzhi Zhang, Haofeng Wang, and Ge Lan. 2026. A Two-Stage GPU Kernel Tuner Combining Semantic Refactoring and Search-Based Optimization. arXiv:2601.12698 <https://arxiv.org/abs/2601.12698>
- [49] Jonathan Ragan-Kelley, Connelly Barnes, Andrew Adams, Sylvain Paris, Frédo Durand, and Saman Amarasinghe. 2013. Halide: A Language and Compiler for Optimizing Parallelism, Locality, and Recomputation in Image Processing Pipelines. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation*. <https://people.csail.mit.edu/jrk/halide-pldi13.pdf>
- [50] Eric Schkufza, Rahul Sharma, and Alex Aiken. 2013. Stochastic Superoptimization. In *Proceedings of the Eighteenth International Conference on Architectural Support for Programming Languages and Operating Systems*. doi:10.1145/2451116.2451150
- [51] Taras Sereda, Burak Bartan, Ankita Nayak, Tom St.John, Natalie Serrino, and Zain Asgar. 2026. KForge: LLM-Driven Cross-Platform Kernel Generation for AI Accelerators. arXiv:2606.02963 [cs.LG] <https://arxiv.org/abs/2606.02963>
- [52] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, Vol. 36. 8634–8652. doi:10.52202/075280-0377
- [53] Alexander Shypula, Aman Madaan, Yimeng Zeng, Uri Alon, Jacob Gardner, Yiming Yang, Milad Hashemi, Graham Neubig, Parthasarathy Ranganathan, Osbert Bastani, and Amir Yazdanbakhsh. 2024. Learning Performance-Improving Code Edits. In *International Conference on Learning Representations*. 47277–47301. https://proceedings.iclr.cc/paper_files/paper/2024/hash/ce65173b994cf7c925c71b482ee14a8d-Abstract-Conference.html
- [54] Songqiao Su, Xiaoya Li, Albert Wang, Guoyin Wang, Jiwei Li, and Chris Shum. 2025. CUDA-L2: Surpassing cuBLAS Performance for Matrix Multiplication through Reinforcement Learning. arXiv:2512.02551 <https://arxiv.org/abs/2512.02551>
- [55] Annabelle Sujun Tang, Christopher Priebe, Rohan Mahapatra, Lianhui Qin, and Hadi Esmaeilzadeh. 2025. REASONING COMPILER: LLM-Guided Optimizations for Efficient Model Serving. In *Advances in Neural Information Processing Systems*, Vol. 38. 106904–106930. doi:10.52202/085713-3565
- [56] TileWeave. [n. d.]. TileWeave Project Website. <https://tileweave.github.io>.

- [57] Philippe Tillet, H. T. Kung, and David Cox. 2019. Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*. doi:10.1145/3315508.3329973
- [58] Mircea Trofin, Yundi Qian, Eugene Brevdo, Zinan Lin, Krzysztof Choromanski, and David Li. 2021. MLGO: a Machine Learning Guided Compiler Optimizations Framework. arXiv:2101.04808 <https://arxiv.org/abs/2101.04808>
- [59] Nicolas Vasilache, Oleksandr Zinenko, Theodoros Theodoridis, Priya Goyal, Zachary DeVito, William S. Moses, Sven Verdoolaege, Andrew Adams, and Albert Cohen. 2018. Tensor Comprehensions: Framework-Agnostic High-Performance Machine Learning Abstractions. arXiv:1802.04730 <https://arxiv.org/abs/1802.04730>
- [60] Jinghao Wang, Qiqi Gu, Chenpeng Wu, Jianguo Yao, Haibing Guan, and Xijun Li. 2026. HIERA: Workload-Aware Planning Across Implementation Spaces for GPU Kernel Optimization. arXiv:2608.21157 <https://arxiv.org/abs/2608.21157>
- [61] Lei Wang, Yu Cheng, Yining Shi, Zhiwen Mo, Zhengju Tang, Wenhao Xie, Tong Wu, Lingxiao Ma, Yuqing Xia, Jilong Xue, Fan Yang, and Zhi Yang. 2026. TileLang: Bridge Programmability and Performance in Modern Neural Kernels. In *International Conference on Learning Representations*. 73099–73134. https://proceedings.iclr.cc/paper_files/paper/2026/hash/76fb92288bf90360c527efb0d1c2aba6-Abstract-Conference.html
- [62] Zhongzhen Wen, Shudi Shao, Zhong Li, Yu Ge, Tongtong Xu, Yuanyi Lin, and Tian Zhang. 2026. AscendCraft: Automatic Ascend NPU Kernel Generation via DSL-Guided Transcompilation. arXiv:2601.22760 <https://arxiv.org/abs/2601.22760>
- [63] Zhongzhen Wen, Yinghui Zhang, Zhong Li, Zhongxin Liu, Linna Xie, and Tian Zhang. 2026. MultiKernelBench: A Multi-Platform Benchmark for Kernel Generation. In *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering*. <https://conf.researchr.org/track/ase-2026/ase-2026-research-track> To appear.
- [64] R. Clint Whaley and Jack Dongarra. 1998. Automatically Tuned Linear Algebra Software. In *Proceedings of the ACM/IEEE Conference on Supercomputing*. <https://math-atlas.sourceforge.net/faq.html>
- [65] Max Willsey, Chandrakana Nandi, Yisu Remy Wang, Oliver Flatt, Zachary Tatlock, and Pavel Panchekha. 2021. egg: Fast and extensible equality saturation. *Proceedings of the ACM on Programming Languages* 5, POPL, Article 23 (2021), 29 pages. doi:10.1145/3434304
- [66] Shuxiao Xie, Shuyang Xie, Dezhi Ran, Wei Yang, and Tao Xie. 2026. AKO: Agentic Kernel Optimization. <https://tongminglaic.github.io/AKO>. Technical report.
- [67] Lingyun Yang, Yuxiao Wang, Shenghao Liang, Linfeng Yang, Daocheng Ying, Chunbo You, Rui Zhang, Luping Wang, Yinghao Yu, Guodong Yang, and Liping Zhang. 2026. Are LLM-Generated GPU Kernels Production-Ready? A Trace-Driven Benchmark and Optimization Agent. arXiv:2607.14541 [cs.AI] <https://arxiv.org/abs/2607.14541>
- [68] Zihao Ye, Yingyi Huang, Hongyi Jin, Bohan Hou, Junru Shao, Zhongming Yu, Jinqi Chen, Meghan Cowan, Shiyi Cao, Shanli Xing, Hanfeng Chen, Vinod Grover, Tianqi Chen, and Luis Ceze. 2026. CAKE: Compiler-Agent Co-Design for Frontier Kernel Evolution. arXiv:2608.12629 <https://arxiv.org/abs/2608.12629>
- [69] Sharareh Younesian, Wenwen Ouyang, Sina Rafati, Mehdi Rezagholizadeh, Sharon Zhou, Ji Liu, Yue Liu, Yuchen Yang, Hao Li, Ziqiong Liu, Dong Li, Vikram Appia, Zhenyu Gu, and Emad Barsoum. 2026. AgentKernelArena: Generalization-Aware Benchmarking of GPU Kernel Optimization Agents. arXiv:2605.16819 [cs.CL] <https://arxiv.org/abs/2605.16819>
- [70] Genghan Zhang, Shaowei Zhu, Anjiang Wei, Zhenyu Song, Allen Nie, Zhen Jia, Nandita Vijaykumar, Yida Wang, and Kunle Olukotun. 2026. AccelOpt: A Self-Improving LLM Agentic System for AI Accelerator Kernel Optimization. In *Proceedings of Machine Learning and Systems*, Vol. 8. 541–568. https://proceedings.mlsys.org/paper_files/paper/2026/hash/0f8426558905746fc38da5e335700aec-Abstract-Conference.html
- [71] Lianmin Zheng, Chengfan Jia, Minmin Sun, Zhao Wu, Cody Hao Yu, Ameer Haj-Ali, Yida Wang, Jun Yang, Danyang Zhuo, Koushik Sen, Joseph E. Gonzalez, and Ion Stoica. 2020. Ansor: Generating High-Performance Tensor Programs for Deep Learning. In *14th USENIX Symposium on Operating Systems Design and Implementation*. <https://www.usenix.org/conference/osdi20/presentation/zheng>
- [72] Xinguo Zhu, Shaohui Peng, Jiaming Guo, Yunji Chen, Qi Guo, Yuanbo Wen, Hang Qin, Ruizhi Chen, Qirui Zhou, Ke Gao, Yanjun Wu, Chen Zhao, and Ling Li. 2026. QiMeng-Kernel: Macro-Thinking Micro-Coding Paradigm for LLM-Based High-Performance GPU Kernel Generation. *Proceedings of the AAAI Conference on Artificial Intelligence* 40, 34 (2026), 29168–29176. doi:10.1609/aaai.v40i34.40155